Unsupervised Learning, Clustering & NLP Fundamentals

Complete Study Notes — cleaned, corrected, and organized from the class transcript

Topics: unsupervised learning, customer segmentation, agglomerative and divisive clustering, K-Means, Hierarchical Clustering, dendrograms, DBSCAN, Silhouette Score, NLP, tokenization, stop words, POS tagging, stemming, lemmatization, named entity recognition, BIO tagging, text vectorization, one-hot encoding, Bag of Words, TF-IDF, N-grams, and Word2Vec/CBOW.

1. Unsupervised Learning

Unsupervised learning works primarily with **unlabeled data**. Instead of learning a known target label, the algorithm searches for structure, similarities, groups, lower-dimensional representations, or unusual observations.

After the algorithm discovers patterns, a human may inspect and interpret the resulting groups. Human review does not turn the original training process into supervised learning.

Common use cases: customer segmentation, exploratory pattern discovery, dimensionality reduction, anomaly detection, document grouping, image grouping, and some recommendation-related workflows.

Important correction: anomaly detection is not automatically semi-supervised. It may be unsupervised, semi-supervised, or supervised depending on the available labels and method.

2. Customer Segmentation Example

An e-commerce company may cluster customers using behavior such as purchase frequency, spending, product categories, recency, device preferences, or browsing patterns.

The model can discover groups without manually assigning every customer to a predefined label.

Practical caution: clustering customers simply as “iPhone buyers” and “Android buyers” is already a rule-based segmentation if those categories are explicitly defined. True clustering discovers groups from selected features and a similarity measure.

3. Agglomerative vs Divisive Hierarchical Clustering

Agglomerative clustering is a bottom-up approach. Every observation starts as its own cluster. The closest clusters are repeatedly merged until a stopping condition is reached or all observations belong to one cluster.

Divisive clustering is a top-down approach. All observations begin in one cluster, which is repeatedly divided into smaller clusters.

Both approaches create a hierarchy of nested groups.

4. K-Means Clustering

K-Means partitions observations into K clusters by minimizing the within-cluster squared distance to cluster centroids.

Basic algorithm:
1. Choose K.
2. Initialize K centroids.
3. Assign each observation to the nearest centroid.
4. Recalculate each centroid as the mean of the observations assigned to that cluster.
5. Repeat assignment and centroid updates until convergence or another stopping criterion is reached.

The final centroid is the mean position of the observations assigned to that cluster.

Important: K-Means is sensitive to initialization, feature scale, outliers, and cluster geometry. K-Means++ initialization is commonly used to improve starting centroid selection.

5. How to Choose K in K-Means

K is not always chosen only from business need. Domain requirements can define K, but data-driven methods are commonly used.

Common methods:
• Elbow Method: inspect within-cluster sum of squares/inertia and look for diminishing improvement.
• Silhouette Score: compare cluster cohesion and separation.
• Domain knowledge and operational usefulness.
• Stability analysis across different samples or initializations.

There may be no single objectively correct K. A useful clustering solution must be statistically reasonable and practically interpretable.

6. Hierarchical Clustering and Dendrograms

Hierarchical clustering builds nested clusters. In agglomerative clustering, nearby observations or clusters are repeatedly merged according to a **linkage criterion**.

A **dendrogram** is the tree-like visualization of these merges. The height of a merge represents the dissimilarity at which clusters were joined.

Common linkage methods include single linkage, complete linkage, average linkage, and Ward linkage.

To obtain flat clusters, a horizontal cut can be made across the dendrogram. The number of vertical branches intersected by that cut corresponds to the resulting number of clusters.

Important correction: the cut should not simply be placed wherever it creates the “minimum number” of clusters. The choice should consider large linkage-distance jumps, domain needs, stability, and cluster usefulness.

7. DBSCAN

DBSCAN stands for **Density-Based Spatial Clustering of Applications with Noise**. It identifies dense connected regions and can label isolated observations as noise.

Main parameters:
• **eps (ε):** neighborhood radius.
• **min_samples / MinPts:** minimum neighborhood density required for a core point.

Point types:
• **Core point:** has at least MinPts observations in its ε-neighborhood, according to the implementation's counting convention.
• **Border point:** is not a core point but is density-reachable from a core point.
• **Noise point:** is not assigned to a cluster.

A point initially marked as noise can later become a border point if it is found within the neighborhood expansion of a core point.

Strengths: no need to predefine the number of clusters, can discover irregularly shaped clusters, and explicitly identifies noise.

Weaknesses: sensitive to ε and MinPts, feature scaling, and datasets with strongly varying densities.

8. Silhouette Score

The Silhouette Score evaluates how well an observation fits its own cluster compared with the nearest alternative cluster.

For observation i:
a(i) = average distance from i to other observations in its own cluster.
b(i) = smallest average distance from i to observations in another cluster.
s(i) = [b(i) − a(i)] / max(a(i), b(i))

The score ranges from −1 to 1. Values near 1 indicate strong assignment, values near 0 indicate overlap/boundary behavior, and negative values suggest the observation may fit another cluster better.

Important: a high Silhouette Score is useful evidence, not proof that a clustering is meaningful for the business problem.

9. Natural Language Processing (NLP)

Natural Language Processing is the field concerned with enabling computers to process, analyze, and generate human language.

NLP includes tasks such as text classification, sentiment analysis, information extraction, question answering, translation, summarization, search, and conversational systems.

NLU generally refers to language-understanding tasks and is commonly treated as a subset or component of NLP.

Python libraries include NLTK, spaCy, scikit-learn, Hugging Face Transformers, and others.

10. Tokenization

Tokenization breaks text into smaller units called tokens.

Sentence tokenization divides a document into sentences. **Word tokenization** divides text into words or word-like units. Modern language models often use **subword tokenization**, where words may be divided into reusable pieces.

Tokenization is a foundational preprocessing step, but the appropriate tokenizer depends on the model and task.

11. Stop Words

Stop words are frequent function words such as “the,” “is,” and “of” that may contribute limited discriminative information in some traditional NLP tasks.

Removing stop words can reduce vocabulary size and noise for approaches such as Bag of Words.

Important correction: stop words are not always useless. Words such as “not” can completely change meaning, and modern transformer models usually keep stop words because context matters. Stop-word removal should be task-dependent.

12. Part-of-Speech (POS) Tagging

POS tagging assigns grammatical categories to tokens, such as noun, verb, adjective, adverb, pronoun, or preposition.

Example: in “I am eating food,” a POS tagger identifies grammatical roles for “I,” “am,” “eating,” and “food.”

POS information can support lemmatization, syntactic analysis, information extraction, and other NLP tasks.

Important correction: person names are proper nouns, not all nouns; pronouns such as “he” and “she” are not proper nouns.

13. Stemming

Stemming reduces related word forms by applying heuristic rules, often stripping prefixes or suffixes.

Examples may map words such as “computing,” “computed,” and “computer” toward a common stem, although the output may not be a valid dictionary word.

Common stemmers include Porter, Snowball/Porter2, and Lancaster.

Important correction: stemmers differ in aggressiveness; one cannot generally say Lancaster is simply “better” than Porter or Snowball. The appropriate choice depends on the task.

14. Lemmatization

Lemmatization maps an inflected word to a valid dictionary base form called a **lemma**, using vocabulary and often grammatical information.

For example, with the correct POS context, “running” and “ran” may map to “run.”

Lemmatization is usually linguistically cleaner than stemming but may require more resources and processing.

Important correction: “runner” normally remains “runner,” because it is a noun with a different lexical meaning from the verb “run.”

15. Named Entity Recognition (NER)

NER identifies and classifies spans of text into entity categories such as PERSON, ORGANIZATION, LOCATION, DATE, and MONEY.

Modern NER is not merely hard-coded dictionary lookup. Systems may use rules, statistical sequence models, neural networks, or transformer-based contextual models.

Ambiguity remains a challenge. For example, “Apple” may refer to a company or a fruit depending on context. Contextual NER models are specifically designed to use surrounding words to help resolve such ambiguity.

16. BIO Tagging

BIO is a sequence-labeling format used to mark entity spans.

B = beginning of an entity.
I = inside/continuation of the same entity.
O = outside any target entity.

Example: “New York City” as a location may be labeled **B-LOC, I-LOC, I-LOC**.

Important correction: B means beginning of an *entity*, not beginning of a sentence. O means outside an entity, not output.

17. Text Vectorization and Word Embeddings

Machine-learning models require numerical representations of text. **Vectorization** converts text into numeric feature vectors.

Traditional sparse representations include one-hot encoding, Bag of Words, TF-IDF, and N-gram features.

Dense learned representations include Word2Vec, GloVe, FastText, and contextual embeddings from transformer models.

Important correction: vectorization is not simply converting text into binary machine-language 0s and 1s. It represents linguistic units as numeric vectors that algorithms can process.

18. One-Hot Encoding for Words

In one-hot encoding, each vocabulary item is represented by a vector containing one 1 and zeros elsewhere.

Example with vocabulary [Alice, Bob, Charlie]: Alice $\rightarrow$ [1,0,0], Bob $\rightarrow$ [0,1,0], Charlie $\rightarrow$ [0,0,1].

Limitations: high dimensionality, sparse vectors, and no inherent semantic similarity. The representation does not naturally show that two related words are closer in meaning.

19. Bag of Words (BoW)

Bag of Words represents a document using word occurrence counts or binary presence indicators. Word order is ignored.

Advantages: simple, interpretable, and often effective as a baseline.

Limitations: sparse high-dimensional vectors, loss of word order, limited semantic understanding, and potentially excessive influence from frequent words.

Stop-word handling, vocabulary limits, N-grams, and weighting schemes can improve a BoW pipeline.

20. TF-IDF

TF-IDF stands for **Term Frequency–Inverse Document Frequency**.

Term Frequency (TF) measures how frequently a term occurs within a document. A common form is term count divided by total terms in the document.

Inverse Document Frequency (IDF) reduces the weight of terms that occur across many documents. A common unsmoothed form is log(N / df), where N is the number of documents and df is the number containing the term.

TF-IDF = TF × IDF

Important correction: a term appearing in fewer documents receives a *higher* IDF, not lower importance. If a term occurs in every document, its unsmoothed IDF approaches zero. Real libraries often use smoothed formulas.

TF-IDF improves weighting but still does not inherently capture full word order or deep semantic meaning.

21. N-Grams

An N-gram is a contiguous sequence of N tokens.

Unigram: one token.
Bigram: two consecutive tokens.
Trigram: three consecutive tokens.

N-grams preserve limited local word order. For example, the bigram “coffee shop” can become a feature distinct from the individual words “coffee” and “shop.”

Skip-grams allow tokens to be separated by a limited gap rather than requiring strict adjacency.

Trade-off: larger N captures more local context but greatly increases vocabulary sparsity. Rare N-grams may not generalize well.

22. Word2Vec

Word2Vec learns dense vector representations from surrounding-word prediction tasks. Words used in similar contexts can receive nearby vector representations.

The two classic Word2Vec architectures are **CBOW** and **Skip-gram**.

CBOW: predicts a target word from surrounding context words.
Skip-gram: predicts surrounding context words from a target word.

These models use shallow neural-network-style training objectives to learn embeddings. The useful output is the learned embedding matrix.

Important correction: CBOW is not itself “the first neural-network model” for NLP, and Word2Vec does not directly solve all word-order/context problems. Standard Word2Vec produces one static vector per word, so different meanings of the same word can still be conflated.

23. Traditional NLP vs Modern Contextual Embeddings

Traditional count-based methods such as BoW and TF-IDF are strong baselines for many classification and retrieval tasks.

Static embeddings such as Word2Vec capture distributional similarity but assign a single vector to each vocabulary item.

Modern transformer models such as BERT generate **contextual embeddings**: the representation of a word changes according to its surrounding text. This helps distinguish meanings such as “bank” in a financial context versus a river-bank context.

More advanced does not automatically mean better. For small datasets, simple TF-IDF plus a linear classifier can be faster, cheaper, and surprisingly competitive.

24. Practical Clustering Workflow

1. Define the business objective and what a useful cluster means.
2. Select meaningful features.
3. Handle missing values and outliers.
4. Scale features when distance-based algorithms require it.
5. Try a suitable algorithm: K-Means, hierarchical clustering, DBSCAN, etc.
6. Tune algorithm-specific parameters.
7. Evaluate using internal metrics such as Silhouette Score and stability checks.
8. Visualize carefully, possibly using PCA or another dimensionality-reduction method for display.
9. Interpret clusters with domain knowledge.
10. Validate whether the clusters support real decisions.

25. Practical Traditional NLP Workflow

1. Define the NLP task.
2. Clean obvious noise without destroying useful context.
3. Tokenize appropriately.
4. Decide whether stop-word removal, stemming, or lemmatization is useful.
5. Convert text to features using TF-IDF, N-grams, embeddings, or contextual representations.
6. Train the downstream model.
7. Evaluate with task-appropriate metrics.
8. Perform error analysis rather than relying only on one aggregate score.

26. Interview-Ready Answers

What is unsupervised learning? Learning patterns or structure from data without predefined target labels.

How does K-Means work? It repeatedly assigns observations to the nearest centroid and updates centroids to the mean of their assigned observations until convergence.

K-Means vs DBSCAN? K-Means requires K and favors centroid-based compact clusters; DBSCAN uses density, can find irregular shapes, and identifies noise.

What is a dendrogram? A hierarchical clustering tree showing the order and dissimilarity level at which clusters merge.

What is the Silhouette Score? A measure comparing how close a point is to its own cluster versus the nearest alternative cluster.

Stemming vs lemmatization? Stemming applies heuristic reductions and may produce nonwords; lemmatization aims for a valid dictionary base form using linguistic information.

BoW vs TF-IDF? BoW uses occurrence counts; TF-IDF downweights terms common across many documents.

What are N-grams? Consecutive sequences of N tokens used to capture limited local order.

CBOW vs Skip-gram? CBOW predicts a target from context; Skip-gram predicts context from a target.

Quick Comparison Tables

Algorithm	Main Idea	Need K?	Handles Noise?
K-Means	Centroid-based partitioning	Yes	Not explicitly
Hierarchical	Nested merge/split hierarchy	Not initially	Not explicitly
DBSCAN	Density-connected regions	No	Yes

Representation	Captures Frequency?	Captures Local Order?	Dense?
One-Hot	No	No	No
Bag of Words	Yes	No	No
TF-IDF	Weighted	No	No
N-Grams	Yes	Limited	No
Word2Vec	Learns from context	Indirectly	Yes

Recommended study order: Unsupervised Learning → K-Means → choosing K → Hierarchical Clustering → dendrograms → DBSCAN → Silhouette Score → NLP → tokenization → stop words → POS → stemming → lemmatization → NER/BIO → one-hot → BoW → TF-IDF → N-grams → Word2Vec/CBOW.